University of West Attica
School of
Engineering Department of Informatics and Computer Engineering

Digital Systems Design Laboratory

Lab Exercise 3: Getting to know the simulation environment

1. Getting to know the tool (1)
Following the instructions in the file "Simulation in the Modelsim Altera Starter edition 6_uniwa tool" (Part A), implement and simulate a 2-in-1 multiplexer circuit.

2. Getting to know the tool (2)
Following the instructions in the file "Simulation in the Altera Modelsim Starter edition 6_uniwa tool" (Part B), implement and simulate a 2-in-1 multiplexer circuit using testbench

3. 2-in-1 triple multiplexer
Implement a 2-in-1 triple multiplexer circuit, which takes two 3-bit inputs a and b, a selection input s, and outputs a 3-bit signal d, depending on whether input s is 1 or 0. The entity description is as follows:

```
entity mux_double_2to1 is
port
 (a, b: in std_logic_vector(2 downto 0);
        s: in std_logic;
        d: out std_logic_vector(2 downto 0));
end mux_double_2to1;
```

Simulate the circuit for the following input combinations:

s	a	b	d
0	001	010	
0	010	100	
0	111	011	
0	101	111	
1	010	001	
1	000	101	
1	101	010	
1	111	101	

Using tb that you will develop for this purpose.

4. 4-in-1 Multiplexer A 4-
in-1 multiplexer takes a 4-digit data input and a 2-digit control input and has a single-digit input which is either of the data inputs depending on the combination of values of the select input. The entity description is as follows:

```
entity mux_4to1 is port ( a: in
        std_logic_vector(4 downto 1);
        s: in std_logic_vector(2 downto 1);
        d: out std_logic);
end mux_4to1;
```

Simulate the circuit for the following input combinations:

a	s	d
0000	00	

0101	01	
1010	10	
1100	11	

5. 2-to-4 Decoder Implement a
2-to-4 decoder circuit. The entity description is as follows:

```
entity dec2to4 is
port (
  a: in std_logic_vector(2 downto 1);
  d: out std_logic_vector(4 downto 1));
end dec2to4;
```

Implement the circuit using only the <= command (based on the logic circuit)

Check the circuit for the following combinations using force commands:

a	d
00	
01	
10	
11	

6. 2-in-4 decoder with permission

Implement a 2-to-4 decoder circuit with permission. If the permission input is 0, all outputs are 0. The entity description is as follows:

```
entity dec_2to4 is
port (
  a: in std_logic_vector(2 downto 1);
  en: in std_logic;
  d: out std_logic_vector(4 downto 1) );
end dec2to4;
```

Implement the circuit using only the <= command (based on the logic circuit)

Check the circuit for the following combinations

a	en	d
00	0	
01	0	
10	0	
11	0	
00	1	
01	1	
10	1	
11	1	

7. 4-to-16 decoder

Implement a 4-to-16 decoder circuit. The entity description is as follows:

```
entity dec_4to16 is port (
  a: in std_logic_vector( 4 downto 1);
  d: out std_logic_vector(16 downto 1));
end dec4to16;
```

Implement the circuit using only the <= command (based on the logic circuit)

Check the circuit for the following combinations with force commands

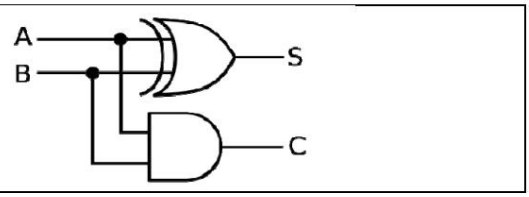
a	d
0000	
0001	
0010	
0011	
0100	
0101	
0110	
0111	
1000	
1001	
1010	
1011	
1100	
1101	
1110	
1111	

8. Half-adder

Implement a half-adder circuit. The entity description is as follows:

```
entity ha is port ( A, B : in bit;

  S,C : out bit);
end huh?
```



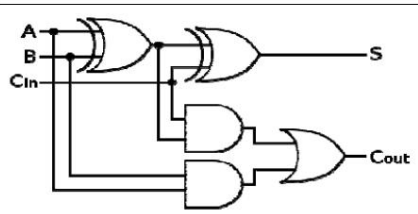
Check the circuit for the following combinations:

A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

9. Full adder

Implement a full adder circuit. The entity description is as follows:

```
entity fa is port (
  A, B, Cin : in bits;
  S, Cout : out bit);
end fa?
```



Check the circuit for the following combinations A

	B	Cin	Cout	S
0	0	0	0	0
0	0	1	0	1

0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

10. 4-bit adder

Implement a 4-bit adder circuit. The entity description is as follows:

```
ENTITY adder4 IS PORT ( Cin : IN
  STD_LOGIC;
  X, Y : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
  S : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
  Cout : OUT STD_LOGIC);
END adder4;
```

Check the circuit for the following combinations A

	B	Cin	Cout	S
0000	0000	0	0	0000
1111	1111	0	1	1110
1111	1111	1	1	1111

Then check the circuit for adding the following numbers

- 3 + 5
- -2 + (3)
- -8 + (+7)